

Integrantes: Ezequiel Daher; Alex Claros; Bautista Quesada.

Materia: Programación Orientada a Objetos.

Carrera: Tec. Desarrollo de Software.

Patrones **GRASP** aplicados

En esta segunda entrega se incorporó una arquitectura en capas con repositorios, servicios, controladores y vistas por consola. La idea fue separar mejor las responsabilidades respecto de la entrega 1 y evitar que la lógica de negocio quede mezclada con la entrada de datos por consola.

El patrón **Controller** se ve en los controladores y en el menú principal, que reciben la acción del usuario y delegan la operación correspondiente. La vista se limita a pedir y mostrar datos, mientras que el controller coordina el flujo entre vista y servicio.

El patrón **Expert** se aplicó asignando responsabilidades a las clases que tienen la información necesaria. Por ejemplo, cada subtipo de odontólogo conoce su tarifa base, su especialidad y qué motivos de consulta puede atender. También el repositorio de turnos es el que conoce la colección de turnos cargados y, por eso, es el lugar adecuado para consultar conflictos de horario.

El patrón **Creator** aparece en la creación de objetos dentro del flujo del sistema. Los turnos no se crean directamente desde la vista, sino desde la capa que ya dispone de paciente, odontólogo, secretaria, fecha, hora y motivo validados. De la misma manera, en el caso de odontólogos se parte de un dato general y luego se crea la subclase concreta correspondiente según la especialidad elegida.

Principios **SOLID** demostrados

El principio más claro en el proyecto es **SRP**. Cada capa tiene una responsabilidad específica: los repositorios guardan y recuperan datos en memoria, los servicios validan y aplican reglas de negocio, las vistas manejan la interacción por consola y los controladores conectan esas capas. Además, el cálculo del monto a pagar no quedó repartido en varias clases, sino concentrado en Facturador, para no sobrecargar otras clases con una responsabilidad extra.

También se aplicó **OCP**. El mejor ejemplo es la jerarquía de odontólogos: el sistema puede incorporar nuevas especialidades agregando nuevas subclases sin modificar la estructura general del modelo. Algo parecido ocurre con la interfaz genérica de repositorio, que permite mantener una base común y luego especializar el comportamiento de cada repositorio según la entidad que administra.

Decisiones de herencia y polimorfismo

La principal decisión de herencia fue mantener una clase base **Persona** para agrupar atributos comunes como id, nombre, apellido y dni. Esto evitó repetir esos datos en Paciente, Secretaria y Odontólogo, y ayudó a ordenar mejor el modelo.

La herencia se aprovechó también en **Odontologo**, donde se definió una jerarquía con OdontologoGeneral, Ortodoncista y Endodoncista.

El **polimorfismo** se refleja en métodos como getTarifaBase() y puedeAtender(...). Cada subtipo responde de manera distinta según su especialidad. Esto permitió evitar validaciones manuales dispersas y dejar que cada objeto resuelva su comportamiento según su propio tipo. De esa manera, la capa de servicios puede trabajar con la abstracción Odontologo sin perder el comportamiento específico de cada especialidad.